**machine learning Project **

Loan Default Prediction system.

```
import pandas as pd
import matplotlib as plt
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
```

```
df = pd.read_csv("/content/lc_2016_2017.csv")
```

```
/tmp/ipython-input-2225036377.py:1: DtypeWarning: Columns (18,54) have mixed types. Specify dtype option on import or set lo
  df = pd.read_csv("/content/lc_2016_2017.csv")
```

```
df.head()
```

	id	member_id	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	sub_grade	...	total
0	112435993	NaN	2300	2300	2300.0	36 months	12.62	77.08	C	C1	...	
1	112290210	NaN	16000	16000	16000.0	60 months	12.62	360.95	C	C1	...	
2	112436985	NaN	6025	6025	6025.0	36 months	15.05	209.01	C	C4	...	
3	112439006	NaN	20400	20400	20400.0	36 months	9.44	652.91	B	B1	...	
4	112438929	NaN	13000	13000	13000.0	36 months	11.99	431.73	B	B5	...	

5 rows × 72 columns

```
df.shape
```

```
(759338, 72)
```

I converted this into a binary classification problem:

0 → Good loan (Fully Paid, Current)

1 → Bad loan (Late, Charged Off, Default)”

```
df.columns
```

```
Index(['id', 'member_id', 'loan_amnt', 'funded_amnt', 'funded_amnt_inv',
       'term', 'int_rate', 'installment', 'grade', 'sub_grade', 'emp_title',
       'emp_length', 'home_ownership', 'annual_inc', 'verification_status',
       'issue_d', 'loan_status', 'pymnt_plan', 'desc', 'purpose', 'title',
       'zip_code', 'addr_state', 'dti', 'delinq_2yrs', 'earliest_cr_line',
       'inq_last_6mths', 'mths_since_last_delinq', 'mths_since_last_record',
       'open_acc', 'pub_rec', 'revol_bal', 'revol_util', 'total_acc',
       'initial_list_status', 'out_prncp', 'out_prncp_inv', 'total_pymnt',
       'total_pymnt_inv', 'total_rec_prncp', 'total_rec_int',
       'total_rec_late_fee', 'recoveries', 'collection_recovery_fee',
       'last_pymnt_d', 'last_pymnt_amnt', 'next_pymnt_d', 'last_credit_pull_d',
       'collections_12_mths_ex_med', 'mths_since_last_major_derog',
       'policy_code', 'application_type', 'annual_inc_joint', 'dti_joint',
       'verification_status_joint', 'acc_now_delinq', 'tot_coll_amt',
       'tot_cur_bal', 'open_acc_6m', 'open_il_12m', 'open_il_24m',
       'mths_since_rcnt_il', 'total_bal_il', 'il_util', 'open_rv_12m',
       'open_rv_24m', 'max_bal_bc', 'all_util', 'total_rev_hi_lim', 'inq_fi',
       'total_cu_tl', 'inq_last_12m'],
      dtype='object')
```

**Droping Columns **

```
df = df.drop(columns = [
    "id",
    "funded_amnt",
    "funded_amnt_inv",
    "pymnt_plan",
    "out_prncp",
    "out_prncp_inv",
    "total_rec_prncp",
    "total_rec_int",
    "total_rec_late_fee",
    "collections_12_mths_ex_med",
    "policy_code",
    "acc_now_delinq",
    "tot_coll_amt",
    "tot_cur_bal",
    "open_acc_6m",
    "open_il_12m",
    "open_il_24m",
    "open_rv_12m",
    "open_rv_24m",
    "max_bal_bc",
    "all_util",
    "total_bal_il",
    "total_rev_hi_lim",
    "inq_fi",
    "total_cu_tl",
    "member_id",
    "title",
    "zip_code",
    "earliest_cr_line",
    "addr_state",
    "emp_title",
    "desc",
    "next_pymnt_d",
    "last_pymnt_d",
    "last_credit_pull_d",
    "total_pymnt",
    "total_pymnt_inv",
    "last_pymnt_amnt",
    "recoveries",
    "collection_recovery_fee",
    "issue_d",
    "annual_inc_joint",
    "dti_joint",
    "verification_status_joint",
    "mths_since_last_delinq",
    "mths_since_last_record",
    "mths_since_last_major_derog",
    "mths_since_rcnt_il",
    "il_util",
    "inq_last_12m"
])
```

```
df.columns
```

```
Index(['loan_amnt', 'term', 'int_rate', 'installment', 'grade', 'sub_grade',
       'emp_length', 'home_ownership', 'annual_inc', 'verification_status',
       'loan_status', 'purpose', 'dti', 'delinq_2yrs', 'inq_last_6mths',
       'open_acc', 'pub_rec', 'revol_bal', 'revol_util', 'total_acc',
       'initial_list_status', 'application_type'],
      dtype='object')
```

```
# df = df.drop(columns="inq_last_12m")
```

```
df.columns
```

```
Index(['loan_amnt', 'term', 'int_rate', 'installment', 'grade', 'sub_grade',
       'emp_length', 'home_ownership', 'annual_inc', 'verification_status',
       'loan_status', 'purpose', 'dti', 'delinq_2yrs', 'inq_last_6mths',
       'open_acc', 'pub_rec', 'revol_bal', 'revol_util', 'total_acc',
       'initial_list_status', 'application_type'],
      dtype='object')
```

```
df.head()
```

	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_length	home_ownership	annual_inc	verification_status
0	2300	36 months	12.62	77.08	C	C1	NaN	OWN	10000.0	Not Verified
1	16000	60 months	12.62	360.95	C	C1	10+ years	MORTGAGE	94000.0	Not Verified
2	6025	36 months	15.05	209.01	C	C4	7 years	MORTGAGE	46350.0	Not Verified
3	20400	36 months	9.44	652.91	B	B1	10+ years	RENT	44000.0	Source Verified
4	13000	36 months	11.99	431.73	B	B5	10+ years	MORTGAGE	85000.0	Source Verified

5 rows × 22 columns

```
df.shape
```

```
(759338, 22)
```

```
Cleaning Data
```

```
df.isna().sum()
```

	0
loan_amnt	0
term	0
int_rate	0
installment	0
grade	0
sub_grade	0
emp_length	50363
home_ownership	0
annual_inc	0
verification_status	0
loan_status	0
purpose	0
dti	355
delinq_2yrs	0
inq_last_6mths	1
open_acc	0
pub_rec	0
revol_bal	0
revol_util	517
total_acc	0
initial_list_status	0
application_type	0

dtype: int64

```
df['emp_length'] = df['emp_length'].replace({"< 1 year" : "0", "10+ years" : "11"})
```

```
df['emp_length']
```

	emp_length
0	NaN
1	11
2	7 years
3	11
4	11
...	...
759333	0
759334	NaN
759335	11
759336	11
759337	7 years

759338 rows × 1 columns

dtype: object

```
df['emp_length'] = df['emp_length'].str.replace("years", "")
df['emp_length'] = df['emp_length'].str.replace("year", "")
```

```
df['emp_length']
```

	emp_length
0	NaN
1	11
2	7
3	11
4	11
...	...
759333	0
759334	NaN
759335	11
759336	11
759337	7

759338 rows × 1 columns

dtype: object

```
df.emp_length.unique()
```

```
array([nan, '11', '7 ', '6 ', '2 ', '8 ', '3 ', '0', '1 ', '9 ', '5 ',
       '4 '], dtype=object)
```

```
sns.boxplot(x=df['emp_length'])
plt.show()
```

Show hidden output

```
df.isna().sum()
```

	0
loan_amnt	0
term	0
int_rate	0
installment	0
grade	0
sub_grade	0
emp_length	50363
home_ownership	0
annual_inc	0
verification_status	0
loan_status	0
purpose	0
dti	355
delinq_2yrs	0
inq_last_6mths	1
open_acc	0
pub_rec	0
revol_bal	0
revol_util	517
total_acc	0
initial_list_status	0
application_type	0

dtype: int64

```
df.info()
```

Show hidden output

```
df['emp_length'] = df['emp_length'].fillna(0)
```

```
from numpy import int64
df['emp_length']=df['emp_length'].astype(int64)
```

```
df['emp_length'].unique()
```

```
array([ 0, 11,  7,  6,  2,  8,  3,  1,  9,  5,  4])
```

```
df.isna().sum()
```

	0
loan_amnt	0
term	0
int_rate	0
installment	0
grade	0
sub_grade	0
emp_length	0
home_ownership	0
annual_inc	0
loan_status	0
purpose	0
dti	358
delinq_2yrs	0
inq_last_6mths	1
open_acc	0
pub_rec	0
revol_bal	0
revol_util	517
total_acc	0
initial_list_status	0
application_type	0

```
df.shape
```

```
(759338, 22)
```

Double-click (or enter) to edit

change base on the columns (dti)

```
df['dti'] = df['dti'].fillna(df['dti'].median())
```

```
df['revol_util'] = df['revol_util'].fillna(df['revol_util'].median())
```

```
df.isna().sum()
```

	0
loan_amnt	0
term	0
int_rate	0
installment	0
grade	0
sub_grade	0
emp_length	0
home_ownership	0
annual_inc	0
verification_status	0
loan_status	0
purpose	0
dti	0
delinq_2yrs	0
inq_last_6mths	1
open_acc	0
pub_rec	0
revol_bal	0
revol_util	0
total_acc	0
initial_list_status	0
application_type	0

dtype: int64

```
df['inq_last_6mths'] = df['inq_last_6mths'].fillna(df['inq_last_6mths'].median())
```

EDA

```
df.info()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 759338 entries, 0 to 759337
Data columns (total 22 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   loan_amnt            759338 non-null  int64
 1   term                 759338 non-null  object
 2   int_rate             759338 non-null  float64
 3   installment          759338 non-null  float64
 4   grade                759338 non-null  object
 5   sub_grade            759338 non-null  object
 6   emp_length           759338 non-null  int64
 7   home_ownership       759338 non-null  object
 8   annual_inc           759338 non-null  float64
 9   verification_status  759338 non-null  object
 10  loan_status          759338 non-null  object
 11  purpose              759338 non-null  object
 12  dti                  759338 non-null  float64
 13  delinq_2yrs          759338 non-null  int64
 14  inq_last_6mths       759338 non-null  float64
 15  open_acc             759338 non-null  int64
 16  pub_rec              759338 non-null  int64
 17  revol_bal            759338 non-null  float64
 18  revol_util           759338 non-null  float64
 19  total_acc            759338 non-null  int64
 20  initial_list_status  759338 non-null  object
 21  application_type     759338 non-null  object
dtypes: float64(7), int64(6), object(9)
memory usage: 127.5+ MB
```

	0
loan_amnt	0
term	0
int_rate	0
installment	0
grade	0
sub_grade	0
emp_length	0
home_ownership	0
annual_inc	0
verification_status	0
loan_status	0
purpose	0
dti	0
delinq_2yrs	0
inq_last_6mths	0
open_acc	0
pub_rec	0
revol_bal	0
revol_util	0
total_acc	0
initial_list_status	0
application_type	0

dtype: int64

```
df.describe()
```

	loan_amnt	int_rate	installment	emp_length	annual_inc	dti	delinq_2yrs	inq_last_6mths
count	759338.000000	759338.000000	759338.000000	759338.000000	7.593380e+05	759338.000000	759338.000000	759338.000000
mean	14707.775260	13.187041	442.584639	5.906965	7.996778e+04	18.980429	0.357795	0.538298
std	9215.456493	5.054311	275.739578	4.303547	1.634141e+05	13.360285	0.951763	0.838306
min	1000.000000	5.320000	30.120000	0.000000	0.000000e+00	-1.000000	0.000000	0.000000
25%	7500.000000	9.750000	241.700000	2.000000	4.800000e+04	12.270000	0.000000	0.000000
50%	12000.000000	12.620000	370.840000	5.000000	6.700000e+04	18.180000	0.000000	0.000000
75%	20000.000000	15.590000	590.505000	11.000000	9.500000e+04	24.770000	0.000000	1.000000

Changing Data types

```
df['term'] = df['term'].str.replace(" months", "").astype(int)
```

```
df['int_rate'] = df['int_rate'].replace("%", "").astype(float)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 759338 entries, 0 to 759337
Data columns (total 22 columns):
 #   Column               Non-Null Count   Dtype  
---  ------               --------------   -----  
 0   loan_amnt            759338 non-null  int64  
 1   term                 759338 non-null  int64  
 2   int_rate             759338 non-null  float64
 3   installment          759338 non-null  float64
 4   grade                759338 non-null  object 
 5   sub_grade            759338 non-null  object 
 6   emp_length           759338 non-null  int64  
 7   home_ownership       759338 non-null  object 
 8   annual_inc           759338 non-null  float64
 9   verification_status  759338 non-null  object 
 10  loan_status          759338 non-null  object 
 11  purpose              759338 non-null  object 
 12  dti                  759338 non-null  float64
 13  delinq_2yrs          759338 non-null  int64  
 14  inq_last_6mths       759338 non-null  float64
 15  open_acc             759338 non-null  int64  
 16  pub_rec              759338 non-null  int64  
 17  revol_bal            759338 non-null  float64
 18  revol_util           759338 non-null  float64
 19  total_acc            759338 non-null  int64  
 20  initial_list_status  759338 non-null  object 
 21  application_type     759338 non-null  object 
dtypes: float64(7), int64(7), object(8)
memory usage: 127.5+ MB
```

**Analysing only Numeric Columns **

focusing only on numeric columns

```
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
num_cols
```

```
Index(['loan_amnt', 'term', 'int_rate', 'installment', 'emp_length',
       'annual_inc', 'dti', 'delinq_2yrs', 'inq_last_6mths', 'open_acc',
       'pub_rec', 'revol_bal', 'revol_util', 'total_acc'],
      dtype='object')
```

```
df[num_cols].describe().T[['min','max']]
```

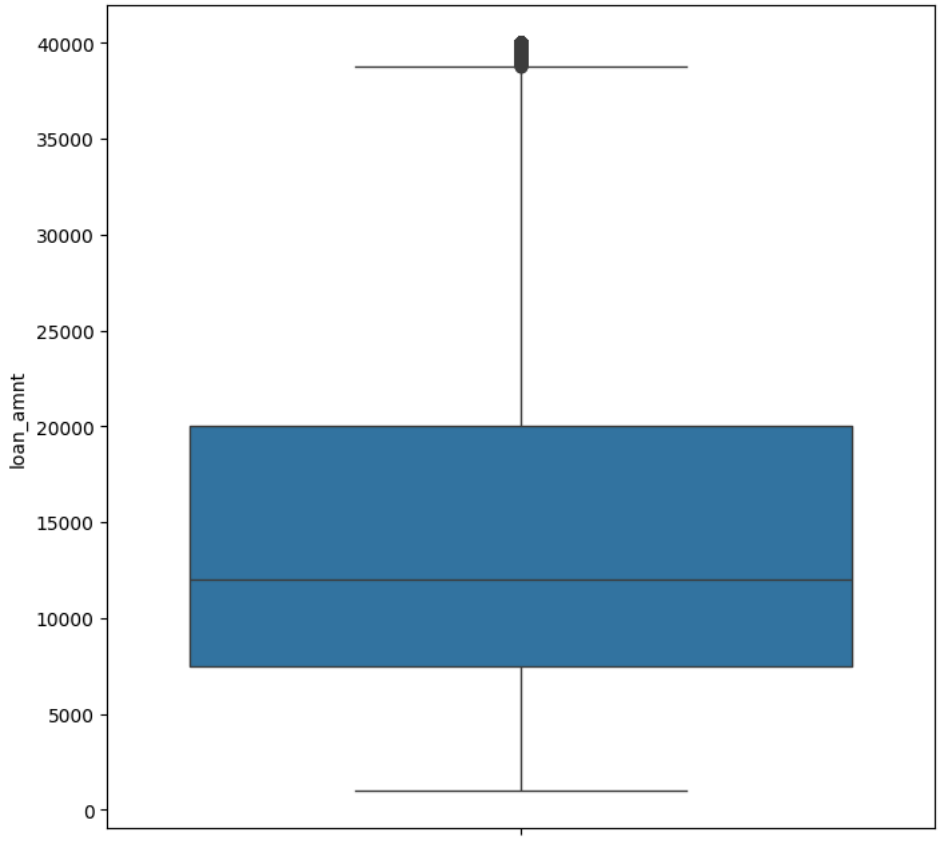
	min	max
loan_amnt	1000.00	4.000000e+04
term	36.00	6.000000e+01
int_rate	5.32	3.099000e+01
installment	30.12	1.719830e+03
emp_length	0.00	1.100000e+01
annual_inc	0.00	1.100000e+08
inq_last_6mths	0.00	5.000000e+00

**BOX plot for Numeric Columns **

```
df['loan_amnt'].value_counts()
```

Show hidden output

```
plt.figure(figsize = (8,8))
sns.boxplot(df['loan_amnt'])
plt.show()
```




```
df['term'].value_counts()
```

term	count
36	561237
60	198101

dtype: int64

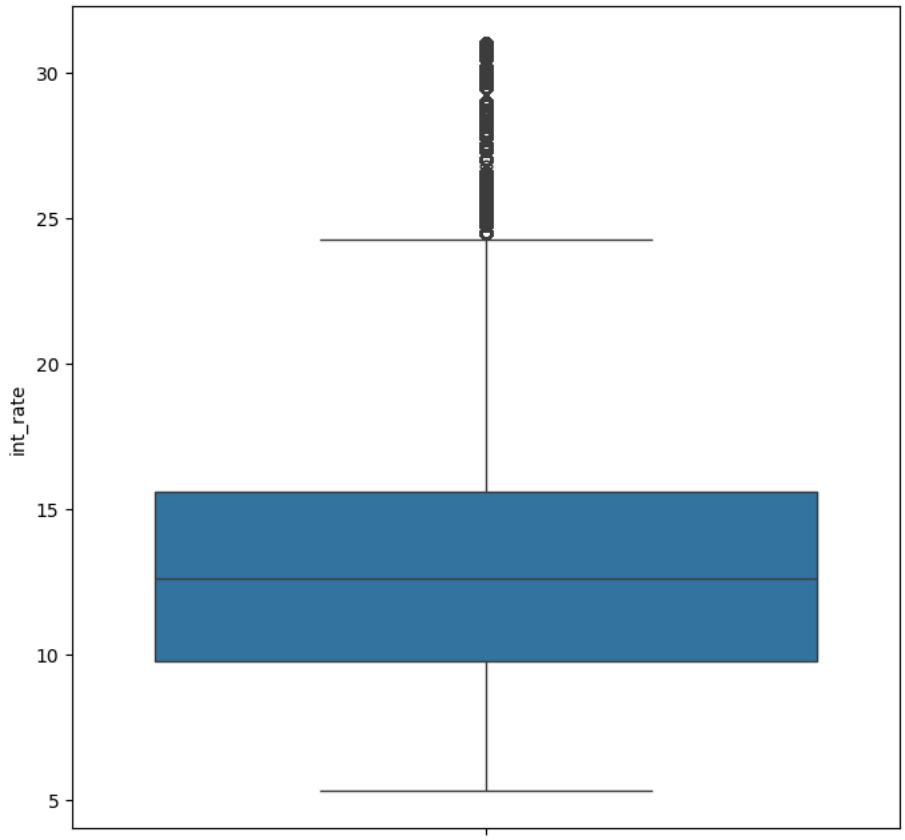
```
plt.figure(figsize = (8,8))
sns.boxplot(df['term'])
plt.show()
```

Show hidden output

```
df['int_rate'].value_counts()
```

Show hidden output

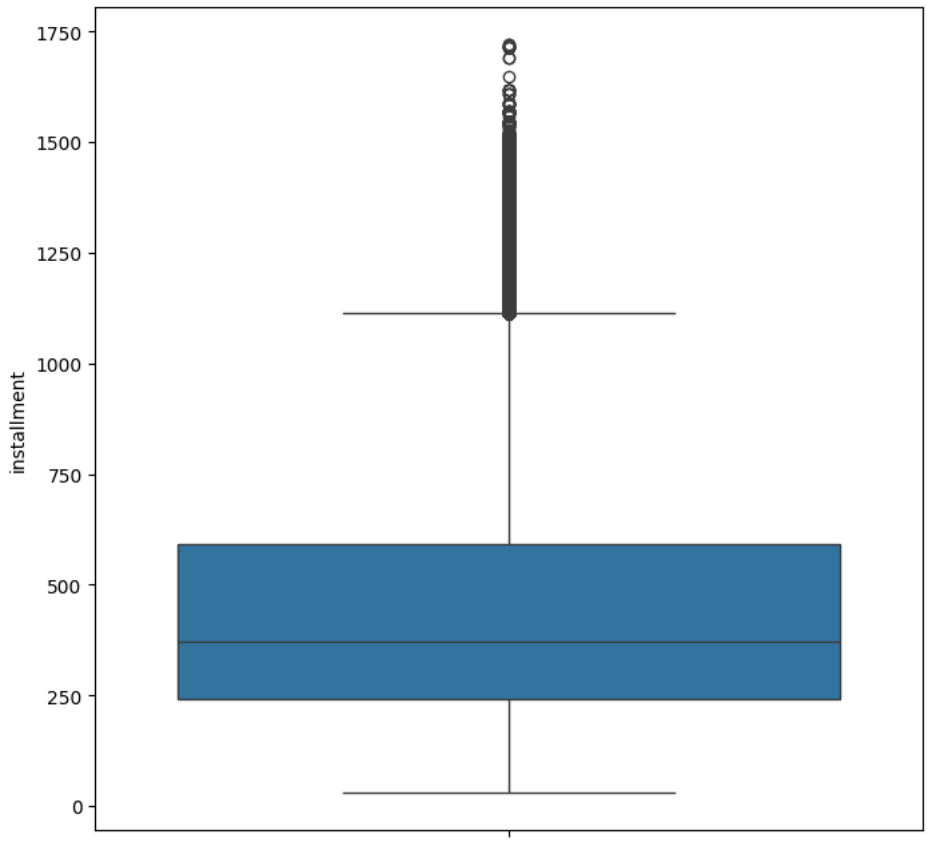
```
plt.figure(figsize = (8,8))
sns.boxplot(df['int_rate'])
plt.show()
```




```
df['installment'].value_counts()
```

Show hidden output

```
plt.figure(figsize = (8,8))
sns.boxplot(df['installment'])
plt.show()
```




```
df['emp_length'].value_counts()
```

Show hidden output

```
df['emp_length'].unique()
```

```
array([ 0, 11,  7,  6,  2,  8,  3,  1,  9,  5,  4])
```

```
plt.figure(figsize = (8,8))
sns.boxplot(df['emp_length'])
plt.show()
```

Show hidden output

```
df['annual_inc'].value_counts()
```

Show hidden output

Remove extremely unrealistic incomes (like 100M)

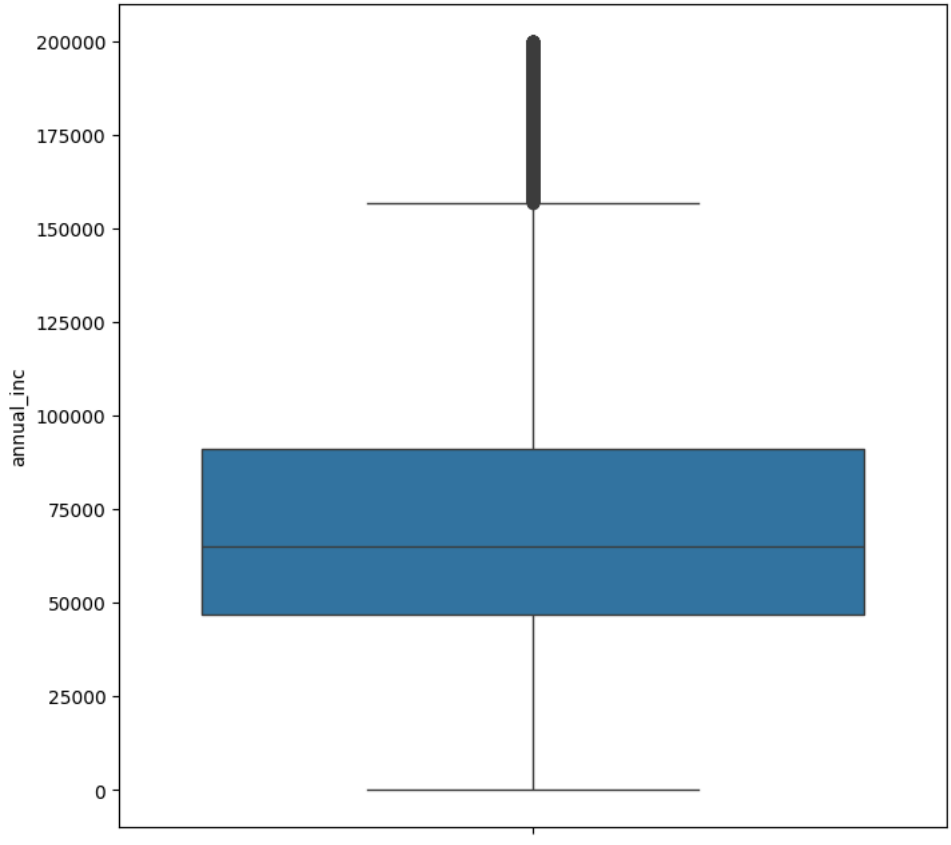
Then apply log transform to smooth the remaining distribution

```
df = df[df['annual_inc'] < 200000]
df['annual_inc_log'] = np.log1p(df['annual_inc'])
```

```
df['annual_inc'].value_counts()
```

Show hidden output

```
plt.figure(figsize = (8,8))
sns.boxplot(df['annual_inc'])
plt.show()
```




```
df['dti'].value_counts()
```

Show hidden output

```
df = df[df['dti'] <= 60]
```

```
plt.figure(figsize = (8,8))
sns.boxplot(df['dti'])
plt.show()
```

Show hidden output

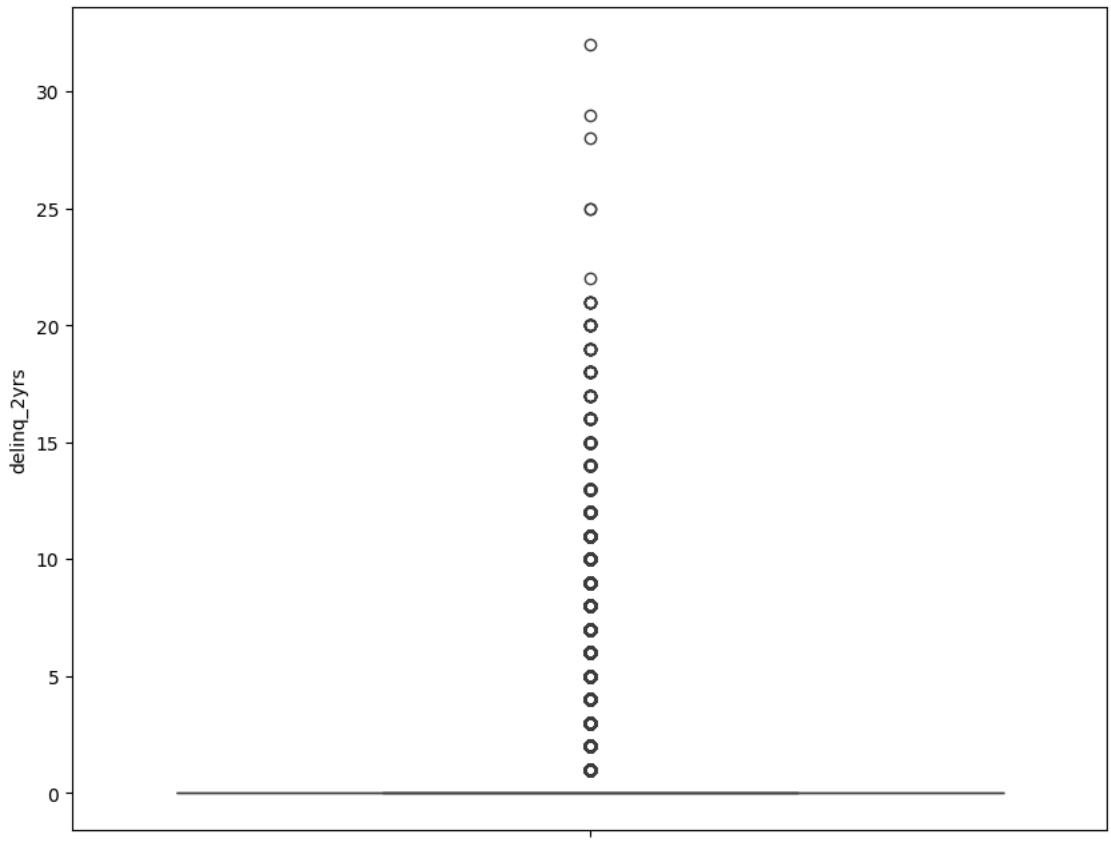
Remove unrealistic values Keep: 0–60 Drop: 60+

```
df['delinq_2yrs'].value_counts()
```

Show hidden output

```
df = df[df['dti'] < 30]
```

```
plt.figure(figsize= (10,8))
sns.boxplot(df['delinq_2yrs'])
plt.show()
```




```
df['inq_last_6mths'].value_counts()
```

inq_last_6mths	count
0.0	410606
1.0	167905
2.0	52861
3.0	16080
4.0	4954
5.0	1728

dtype: int64

```
df['inq_last_6mths'].unique()
```

```
array([1., 0., 2., 3., 4., 5.])
```

```
df['inq_last_6mths'] = df['delinq_2yrs'].fillna(0).astype('int64')
```

```
df = df[df['inq_last_6mths'] < 19]
```

```
plt.figure(figsize=(10,8))
sns.boxplot(df['inq_last_6mths'])
plt.show()
```

Show hidden output

```
df['open_acc'].value_counts()
```

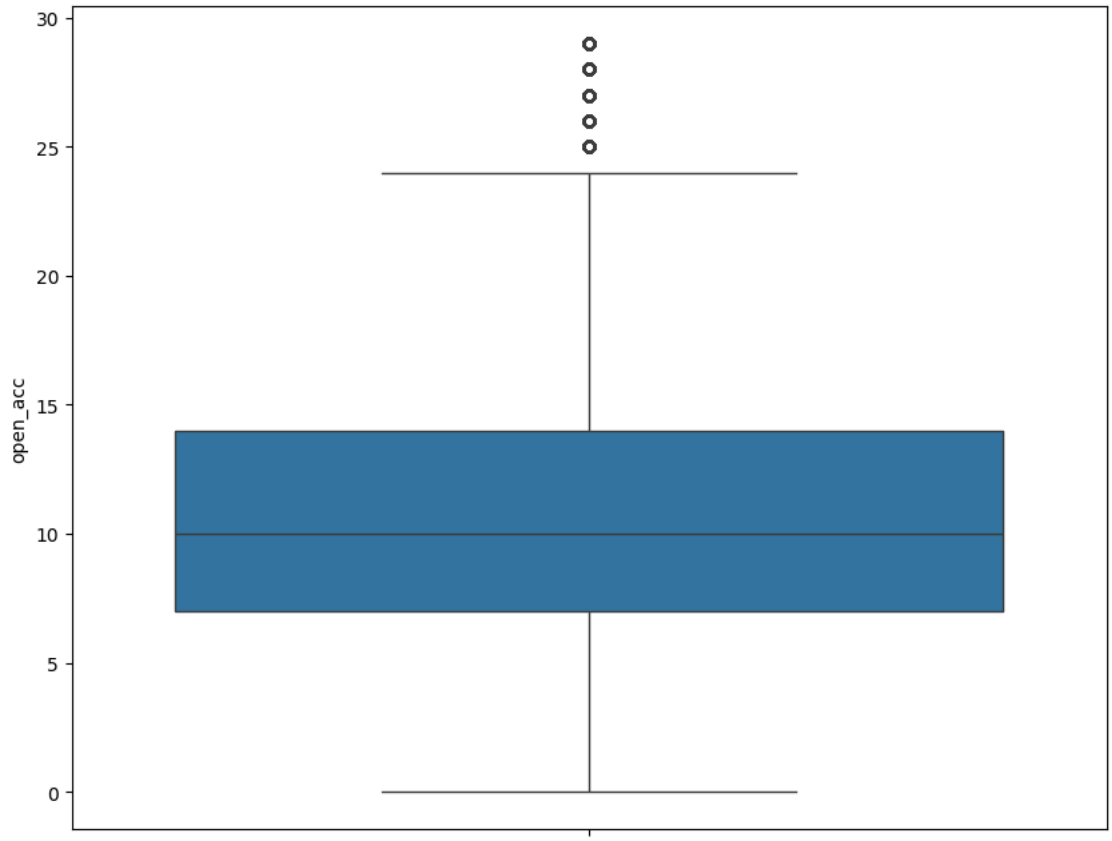
Show hidden output

```
df['open_acc'].unique()
```

Show hidden output

```
df = df[df['open_acc'] < 30]
```

```
plt.figure(figsize=(10,8))
sns.boxplot(df['open_acc'])
plt.show()
```




```
df['pub_rec'].value_counts()
```

Show hidden output

```
df = df[df['pub_rec'] < 12]
```

```
plt.figure(figsize=(10,8))
sns.boxplot(df['pub_rec'])
plt.show()
```

Show hidden output

```
df['revol_bal'].value_counts()
```

Show hidden output

```
df['revol_bal'] = df['revol_bal'].clip(upper=250000)
```

```
plt.figure(figsize=(10,8))
sns.boxplot(df['revol_bal'])
plt.show()
```

Show hidden output

```
df['revol_util'].value_counts()
```

Show hidden output

```
plt.figure(figsize=(10,8))
sns.boxplot(df['revol_util'])
plt.show()
```

Show hidden output

```
df['total_acc'].value_counts()
```

Show hidden output

```
df['total_acc'].unique()
```

Show hidden output

```
df['total_acc'] = df['total_acc'].clip(upper=80)
```

```
plt.figure(figsize=(10,8))
sns.boxplot(df['total_acc'])
plt.show()
```

Show hidden output

```
df['total_acc'].value_counts()
```

Show hidden output

Categorical column

```
df['grade'].value_counts()
```

Show hidden output

```
df['sub_grade'].value_counts()
```

Show hidden output

```
df = df[df['home_ownership'].isin(['MORTGAGE', 'RENT', 'OWN'])]
```

```
df['home_ownership'].value_counts()
```

home_ownership	count
MORTGAGE	311349
RENT	259853
OWN	75763

dtype: int64

```
df['verification_status'].value_counts()
```

verification_status	count
Source Verified	259305
Not Verified	219661
Verified	167999

dtype: int64

```
df['purpose'] = df['purpose'].replace({
    'house': 'home_improvement',
    'vacation': 'other',
    'moving': 'other',
    'renewable_energy': 'other'
})

df = df[df['purpose'] != 'wedding']
```

```
df['purpose'].value_counts()
```

purpose	count
debt_consolidation	365741
credit_card	133639
other	55527
home_improvement	51904
major_purchase	16311
medical	8883
car	7866
small_business	7092

dtype: int64

```
df['initial_list_status'].value_counts()
```

initial_list_status	count
w	497029
f	149934

dtype: int64

```
df['application_type'].value_counts()
```

application_type	count
Individual	625924
Joint App	21039

dtype: int64

Nominal columns (have no order)

1.home ownership 2.verification_status 3.purpose 4.initial_lsit_status 5.application_type

label Encoding for all this columns

```
# from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
df['home_ownership'] = encoder.fit_transform(df['home_ownership'])
df['home_ownership']
```

	home_ownership
0	1
1	0
2	0
3	2
4	0
...	...
759333	1
759334	2
759335	2
759336	0
759337	0

646963 rows × 1 columns

dtype: int64

```
encoder = LabelEncoder()
df['verification_status'] = encoder.fit_transform(df['verification_status'])
df['verification_status']
```

Show hidden output

```
encoder = LabelEncoder()
df['purpose'] = encoder.fit_transform(df['purpose'])
df['purpose']
```

Show hidden output

```
encoder = LabelEncoder()
df['initial_list_status'] = encoder.fit_transform(df['initial_list_status'])
df['initial_list_status']
```

Show hidden output

```
encoder = LabelEncoder()
df['application_type'] = encoder.fit_transform(df['application_type'])
df['application_type']
```

Show hidden output

Ordinal (Has order)

1.grade 2.sub_grade

```
# grade

grade_order = {"A":1, "B":2, "C":3, "D":4, "E":5, "F":6, "G":7}
df["grade_encoded"] = df["grade"].map(grade_order)
df['grade_encoded']
```

	grade_encoded
0	3
1	3
2	3
3	2
4	2
...	...
759333	1
759334	2
759335	3
759336	4
759337	2

646963 rows × 1 columns

dtype: int64

```
# sub_grade
subgrades = [f"{g}{n}" for g in ["A","B","C","D","E","F","G"] for n in range(1,6)] # manually creating all possible loan su
subgrade_order = {sg: i+1 for i, sg in enumerate(subgrades)}

df["sub_grade_encoded"] = df["sub_grade"].map(subgrade_order)
df['sub_grade_encoded']
```

	sub_grade_encoded
0	11
1	11
2	14
3	6
4	10
...	...
759333	5
759334	7
759335	13
759336	16
759337	10

646963 rows × 1 columns

dtype: int64

```
# droping grade and subgrade as we now have grade_encoded and subgrade_encoded
df = df.drop(columns=["grade", "sub_grade"])
```

```
df
```

	loan_amnt	term	int_rate	installment	emp_length	home_ownership	annual_inc	verification_status	loan_status	pu
0	2300	36	12.62	77.08	0	1	10000.0	0	Current	
1	16000	60	12.62	360.95	11	0	94000.0	0	Current	
2	6025	36	15.05	209.01	7	0	46350.0	0	Current	
3	20400	36	9.44	652.91	11	2	44000.0	1	Current	
4	13000	36	11.99	431.73	11	0	85000.0	1	Current	
...	...	...	...	...	...	...	...	...	...	
759333	6000	36	7.89	187.72	0	1	38000.0	1	Current	
759334	6000	36	9.17	191.28	0	2	32640.0	0	Current	
759335	14400	60	13.18	328.98	11	2	47000.0	2	Late (16-30 days)	
759336	34050	36	15.41	1187.21	11	0	87800.0	1	Current	
759337	5000	36	11.22	164.22	7	0	65000.0	1	Current	

646963 rows × 23 columns

```
df['loan_status'].value_counts()
```

loan_status	count
Current	482162
Fully Paid	113674
Charged Off	29679
Late (31-120 days)	12611
In Grace Period	5560
Late (16-30 days)	3249
Default	28

dtype: int64

```
df['loan_status'] = df['loan_status'].replace({
    "Fully Paid": 0,
    "Current": 0,
    "In Grace Period": 1,
    "Late (16-30 days)": 1,
    "Late (31-120 days)": 1,
    "Charged Off": 1,
    "Default": 1
})
```

```
/tmp/ipython-input-639367557.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a f
  df['loan_status'] = df['loan_status'].replace({
```

```
df['loan_status'].value_counts()
```

loan_status	count
0	595836
1	51127

dtype: int64

```
X = df.drop('loan_status', axis=1)
y = df['loan_status']
```

```
X
```

ient	emp_length	home_ownership	annual_inc	verification_status	purpose	dti
7.08	0	1	10000.0	0	1	21.61
0.95	11	0	94000.0	0	2	25.61
9.01	7	0	46350.0	0	3	8.88
2.91	11	2	44000.0	1	0	27.06
1.73	11	0	85000.0	1	2	6.79
...	...	...	...	...	...	...
7.72	0	1	38000.0	1	1	12.35
1.28	0	2	32640.0	0	2	22.76
3.98	11	2	47000.0	2	1	19.64
7.21	11	0	87800.0	1	1	12.10
4.22	7	0	65000.0	1	3	3.10

```
y
```

	loan_status
0	0
1	0
2	0
3	0
4	0
...	...
759333	0
759334	0
759335	1
759336	0
759337	0

646963 rows × 1 columns

dtype: int64

```
print(X.shape, y.shape)
print(y.value_counts())
```

```
(646963, 22) (646963,)
loan_status
0    595836
1     51127
Name: count, dtype: int64
```

Split into Train/Test

```
from sklearn.model_selection import train_test_split

X = df.drop('loan_status', axis=1)
y = df['loan_status']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

- Random Oversampling → Copies same rows (overfitting risk)
- Undersampling → Removes useful data

Appling SMOTE

```
from imblearn.over_sampling import SMOTE

sm = SMOTE(random_state=42)
X_train_sm, y_train_sm = sm.fit_resample(X_train, y_train)
```

Check new counts

```
y_train.value_counts(), y_train_sm.value_counts(),
```

```
(loan_status
 0    476668
 1     40902
 Name: count, dtype: int64,
 loan_status
 0    476668
 1    476668
 Name: count, dtype: int64)
```

**Traning the model **

Logistic Regression

```
# from sklearn.linear_model import LogisticRegression
# from sklearn.metrics import classification_report, accuracy_score

# lr = LogisticRegression(max_iter=500, random_state=42)
# lr.fit(X_train_sm, y_train_sm)

# y_pred_lr = lr.predict(X_test)

# print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
# print(classification_report(y_test, y_pred_lr))
```

Decision Tree

```
# from sklearn.tree import DecisionTreeClassifier

# dt = DecisionTreeClassifier(random_state=42,n_jobs=-1)
# dt.fit(X_train_sm, y_train_sm)

# y_pred_dt = dt.predict(X_test)

# print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
# print(classification_report(y_test, y_pred_dt))
```

Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=50, random_state=42)
rf.fit(X_train_sm, y_train_sm)

y_pred_rf = rf.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.9003346394318085
              precision    recall  f1-score   support

           0       0.92      0.97      0.95    119168
           1       0.18      0.08      0.11     10225

    accuracy                           0.90    129393
   macro avg       0.55      0.52      0.53    129393
weighted avg       0.87      0.90      0.88    129393
```

Gradient Boosting

```
# from sklearn.ensemble import GradientBoostingClassifier

# gb = GradientBoostingClassifier(random_state=42)
# gb.fit(X_train_sm, y_train_sm)

# y_pred_gb = gb.predict(X_test)

# print("Gradient Boosting Accuracy:", accuracy_score(y_test, y_pred_gb))
# print(classification_report(y_test, y_pred_gb))
```

XGBoost

```
# from xgboost import XGBClassifier

# xgb = XGBClassifier(
#     n_estimators=300,
#     learning_rate=0.05,
#     max_depth=6,
#     subsample=0.8,
#     colsample_bytree=0.8,
#     random_state=42
# )

# xgb.fit(X_train_sm, y_train_sm)

# y_pred_xgb = xgb.predict(X_test)

# print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))
# print(classification_report(y_test, y_pred_xgb))
```

thankyou for giving me the opportunity to explain my projerct moving to the application

```

```

```
import joblib
```

```
import pickle
pickle_out = open("Loan_prediction","wb")
pickle.dump(rf,pickle_out)
pickle_out.close()
```

```
import streamlit as st
```

```
def prediction (loan_amnt, term, int_rate, installment, emp_length,
       home_ownership, annual_inc, verification_status,
       purpose, dti, delinq_2yrs, inq_last_6mths, open_acc,
       pub_rec, revol_bal, revol_util, total_acc,
       initial_list_status, application_type, annual_inc_log,
       grade_encoded, sub_grade_encoded):
  prediction = rf.predict([[loan_amnt, term, int_rate, installment, emp_length,
       home_ownership, annual_inc, verification_status,
       purpose, dti, delinq_2yrs, inq_last_6mths, open_acc,
       pub_rec, revol_bal,revol_util, total_acc,
       initial_list_status, application_type, annual_inc_log,
       grade_encoded, sub_grade_encoded]])
```

```
st.title("welcome")
st.write("Enter customer Details")
```

```
home_ownership_map = {
    "RENT": 0,
    "MORTGAGE": 1,
    "OWN": 2
}
```

```
verification_status_map = {
"Source Verified"   : 0,
"Not Verified"  : 1,
"Verified" : 2,
}
```

```
purpose_map= {
"debt_consolidation" : 0,
"credit_card"   : 1,
"other" : 2,
"home_improvement" : 3,
"major_purchase": 4,
"medical"   : 5,
"car"   : 6,
"small_business" : 7
}
```

```
initial_list_status_map = {
"w" : 0,
"f" : 1
}
```

```
application_type_map = {
"Individual" : 0,
"Joint App" : 1
}
```

```
grade_encoded_map = {
"C" : 0,
"B" : 1,
"A" : 2,
"D" : 3,
"E" : 4,
"F" : 5,
"G" : 6
}
```

```
sub_grade_encoded_map = {
"C1" :0,
"B5" :1,
"B4" :2,
"C4" :3,
"C2" :4,
"B1" :5,
"B3" :6,
"C3" :7,
"C5" :8,
"B2" :9,
"A1" :10,
"A5" :11,
"A4" :12,
"D1" :13,
"D2" :14,
"A2" :15,
"A3" :16,
"D3" :17,
"D4" :18,
"D5" :19,
```